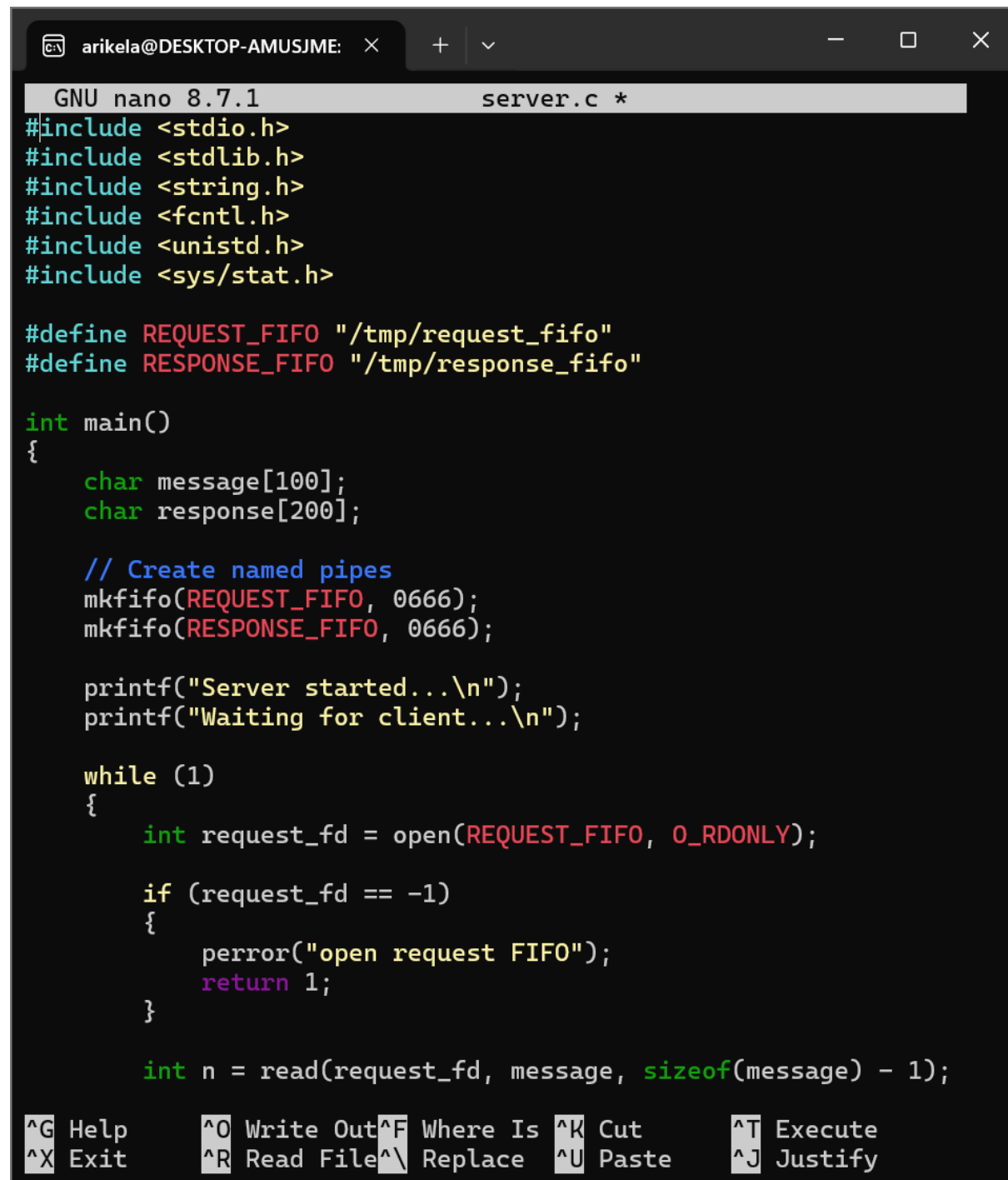


Practical Week-6

Task-1:

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

server.c below:



```
GNU nano 8.7.1 server.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define REQUEST_FIFO "/tmp/request_fifo"
#define RESPONSE_FIFO "/tmp/response_fifo"

int main()
{
    char message[100];
    char response[200];

    // Create named pipes
    mkfifo(REQUEST_FIFO, 0666);
    mkfifo(RESPONSE_FIFO, 0666);

    printf("Server started...\n");
    printf("Waiting for client...\n");

    while (1)
    {
        int request_fd = open(REQUEST_FIFO, O_RDONLY);

        if (request_fd == -1)
        {
            perror("open request FIFO");
            return 1;
        }

        int n = read(request_fd, message, sizeof(message) - 1);

        ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
        ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

arikela@DESKTOP-AMUSJME: ×

+ v

—

□

×

GNU nano 8.7.1

server.c *

```
if (request_fd == -1)
{
    perror("open request FIFO");
    return 1;
}

int n = read(request_fd, message, sizeof(message) - 1);
close(request_fd);

if (n > 0)
{
    message[n] = '\0';

    printf("Server received: %s\n", message);

    if (strcmp(message, "exit") == 0)
    {
        break;
    }

    snprintf(response, sizeof(response),
              "Server processed your message: %s", messa>

    int response_fd = open(RESPONSE_FIFO, O_WRONLY);

    if (response_fd == -1)
    {
        perror("open response FIFO");
        return 1;
    }

    write(response_fd, response, strlen(response) + 1);
```

^G Help

^X Exit

^O Write Out

^R Read File

^F Where Is

^N Replace

^K Cut

^U Paste

^T Execute

^J Justify

```

        write(response_fd, response, strlen(response) + 1);
        close(response_fd);
    }
}

unlink(REQUEST_FIFO);
unlink(RESPONSE_FIFO);

printf("Server stopped.\n");

return 0;
}
|

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]F Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify

client.c below:

GNU nano 8.7.1

client.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define REQUEST_FIFO "/tmp/request_fifo"
#define RESPONSE_FIFO "/tmp/response_fifo"

int main()
{
    char message[100];
    char response[200];

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);

    message[strcspn(message, "\n")] = '\0';

    int request_fd = open(REQUEST_FIFO, O_WRONLY);

    if (request_fd == -1)
    {
        perror("open request FIFO");
        return 1;
    }

    write(request_fd, message, strlen(message) + 1);

    close(request_fd);

    int response_fd = open(RESPONSE_FIFO, O_RDONLY);
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```

write(request_fd, message, strlen(message) + 1);

close(request_fd);

int response_fd = open(RESPONSE_FIFO, O_RDONLY);

if (response_fd == -1)
{
    perror("open response FIFO");
    return 1;
}

int n = read(response_fd, response, sizeof(response) - 1);

if (n > 0)
{
    response[n] = '\0';
    printf("Client received: %s\n", response);
}

close(response_fd);

return 0;
}

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

Terminal 1:

```

arikela@DESKTOP-AMUSJME:~$ mkdir fifo_client_server
arikela@DESKTOP-AMUSJME:~$ cd fifo_client_server
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ pwd
/home/arikela/fifo_client_server
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ nano server.c
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ nano client.c
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ gcc server.c -o server
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ gcc client.c -o client
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ ls
client  client.c  server  server.c
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ ./server
Server started...
Waiting for client...
Server received: hello server

```

Terminal 2:

```

arikela@DESKTOP-AMUSJME:~$ cd fifo_client_server
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ ./client
Enter message: hello server
Client received: Server processed your message: hello server
arikela@DESKTOP-AMUSJME:~/fifo_client_server$

```

Terminal 3:

```
arikela@DESKTOP-AMUSJME: ~$ cd fifo_client_server
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ ./client
Enter message: client 2
Client received: Server processed your message: client 2
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ |
```

Terminal 4:

```
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ ./client
Enter message: client 3
Client received: Server processed your message: client 3
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ |
```

Back to terminal 1:

```
arikela@DESKTOP-AMUSJME:~/fifo_client_server$ ./server
Server started...
Waiting for client...
Server received: hello server
Server received: client 2
Server received: client 3
```

Task-2:

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSRI. Demonstrate asynchronous event handling and explain the role of signal handlers.

GNU nano 8.7.1

signal_handler.c *

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
```

```
void handle_signal(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nReceived SIGINT (Ctrl+C)\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nReceived SIGTERM\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nReceived SIGUSR1\n");
    }

    fflush(stdout);
}
```

```
int main()
{
    // Register signal handlers
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);

    printf("Process ID: %d\n", getpid());
    printf("Signal handling program started...\n");
    printf("Waiting for signals...\n");
}
```

```
^G Help
^X Exit
```

```
^O Write Out
^R Read File
```

```
^F Where Is
^_ Replace
```

```
^K Cut
^U Paste
```

```
^T Execute
^J Justify
```

```

while (1)
{
    printf("Program is running...\n");
    sleep(2);
}

return 0;
}
|

```

^{^G} Help ^{^O} Write Out ^{^F} Where Is ^{^K} Cut ^{^T} Execute
^{^X} Exit ^{^R} Read File ^{^_\} Replace ^{^U} Paste ^{^J} Justify

Terminal 1:

```

arikel@DESKTOP-AMUSJME:~$ cd signal_task
arikel@DESKTOP-AMUSJME:~/signal_task$ pwd
/home/arikel/signal_task
arikel@DESKTOP-AMUSJME:~/signal_task$ nano signal_handler.c
arikel@DESKTOP-AMUSJME:~/signal_task$ gcc signal_handler.c -o s
ignal_handler
arikel@DESKTOP-AMUSJME:~/signal_task$ ls
signal_handler  signal_handler.c
arikel@DESKTOP-AMUSJME:~/signal_task$ ./signal_handler
Process ID: 2571
Signal handling program started...
Waiting for signals...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
^C
Received SIGINT (Ctrl+C)
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...

```

Terminal 2:


```
arikel@DESKTOP-AMUSJME:~$ cd signal_task
arikel@DESKTOP-AMUSJME:~/signal_task$ pgrep signal_handler
2571
arikel@DESKTOP-AMUSJME:~/signal_task$ kill -SIGUSR1 2571
arikel@DESKTOP-AMUSJME:~/signal_task$ kill -SIGTERM 2571
arikel@DESKTOP-AMUSJME:~/signal_task$ kill -9 2571
arikel@DESKTOP-AMUSJME:~/signal_task$
```

Back to terminal 1:

[illegible]

```
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Killed
./signal_handler
arikel@DESKTOP-AMUSJME:~/signal_task$ |
```